

Creative OS Studio — PRD

Clone target: the MaxFusion "Flows" canvas — rebuilt as Creative Abundance's own product, with the one thing MaxFusion doesn't have: an execution engine that runs without a browser.

Prepared July 4, 2026 following the Clone Playbook. Every environment fact below was live-verified this week (not assumed). Hand this document to Fable 5 as the complete build spec.

1. What we're cloning

MaxFusion Flows is a React-Flow-style node canvas where creative pipelines are wired visually: `Image input` → `Content Analyzer` → `Prompt nodes` → `Assistant (Claude)` → `Video Generator (Seedance 2.0)`. The team's tuned master prompts live inside these flows, and their proven ad concepts (15s B-roll, street interview, unboxing, testimonial mashup...) are "the average of" every winning ad in that format.

The core loop we must clone (v1 = the 20% used daily):

1. Client submission arrives from the live website (product name, website, audience, concept, product image)
2. The chosen concept **template flow** is cloned into that client's workspace, inputs injected
3. A **server-side executor** walks the graph: analyzer reads the product image → assistant (creative strategist with the concept library) writes scene prompts → video generator(s) render → outputs saved
4. Dashboard shows **every client, every run, every finished video**; the website gets the video URLs back
5. Winning flows are **saved as templates** and reused for the next client (Ricardo's concept-library model, Eric-approved)

What we are deliberately NOT cloning in v1: full drag-and-drop canvas editing (view + edit input fields only), image/statics pipeline (video first), Notion storyboard auto-distiller, multi-tenant auth, billing/credits UI.

Why clone at all (the business case, from the Jul-3 call): MaxFusion burned ~\$500 of credits in a week; its flow engine only executes while a browser tab is open (verified three times — fatal for a self-serve website); and the concept library belongs in our own product, built from our own clients' wins.

2. Target environment — VERIFIED facts

Live-probed this week. Fable 5 must not re-discover any of this.

MaxFusion API (base `https://api.maxfusion.ai/api/v1` **, header** `Authorization: Bearer <MAXFUSION_API_KEY>` **)**

- `GET /flows` + `GET /flows/{id}` → full `canvas_data` in **React Flow format** (`nodes[{id,type,position,data}]` , `edges`) + `runtime_state` (`node_statuses` , `node_outputs` with S3 URLs, `node_costs` , `last_run_at`). All 34 workspace flows retrievable, master prompts included.
- Flow CRUD works: `POST /flows` (create), `PATCH /flows/{id}` (name/canvas_data — `runtime_state` is **rejected**, read-only), `DELETE /flows/{id}` (204).
- `POST /videos` { `video_model`, `prompt`, `duration:int`, `aspect_ratio`, `resolution` } → { `job_id`, `polling_url` } — **runs fully headless** (proved: real 15s ad end-to-end). **Silently ignores unknown fields** (typos lose data — no `references` / `start_frame` support on the public API).
- `POST /images` { `model`, `prompt`, `aspect_ratio`, `quality`, `references:[file uuids]` } — strict validation, references honored.
- `POST /files` { `purpose:"image_reference"`, `filename`, `content_type`, `size_bytes` } → { `file_id`, `filename`, `upload:{url, fields}` } = S3 presigned **multipart POST** (25 MB cap). **No top-level url** — public URL is `https://maxfusion.s3.eu-west-`

3.amazonaws.com/{filename} .

- `GET /jobs/{job_id}` → `queued` → `running` → `succeeded|failed` ; image ≈ 22s, video ≈ 3–5 min. Occasional spurious `content_policy_violation` on generated audio → retry once, second attempt passed.
- `POST /flows/{id}/run-bulk {node_ids, project_id}` → enqueues only. **Never executes headless** (3 clean-flow tests: `last_run_at` never advanced). Engine is browser-driven. This endpoint must not be used by the executor.
- Models catalog: `GET /models/videos` — `seedance-2.0` : durations 4–15s int, ratios `9_16|1_1|16_9` (underscore format!), resolutions 480p/720p/1080p, audio on by default.

OpenRouter (analyzer + assistant brains)

- `POST https://openrouter.ai/api/v1/chat/completions` , model `anthropic/claude-sonnet-4.6` , `max_tokens: 8000` .
- Vision input: `{type:"image_url", image_url:{url:"data:image/jpeg;base64,..."}}` — the website payload is already in this exact format.
- Response at `choices[0].message.content` ; `**wraps JSON output in ``json fences`` – always strip before JSON.parse``.

Existing working system (do not break)

- Website LIVE: `creative-os-blue.vercel.app` (Vercel project `creative-os` , CLI authed as carlpatriksajol-aidev). Posts flat payload `{product_name, website, target_audience, concept, concept_label, product_images_base64[]}` to `https://n8n-i3t9.srv1486031.hstgr.cloud/webhook/creativeos` , waits up to 12 min, renders any `{videos:[urls]}` in the response as phone-frame previews with download buttons.
- n8n 2.11.3 self-hosted on Hostinger VPS (`srv1486031.hstgr.cloud`) — team's known tool; keeps the public webhook edge.
- Working pipeline ("My workflow 28"): webhook → OpenRouter analyzer → OpenRouter assistant → `POST /videos` → poll loop → respond. This is the reference implementation the executor generalizes.

Build machine (Carl, Windows 11 Pro)

- Node v24.13.0 (npm/npx OK), Python 3.14 + pip, ffmpeg 8.1.1 full, git + Git Bash + PowerShell 5.1, Chrome (headless PDF proven), Vercel CLI authed. No Docker verified — do not assume it.

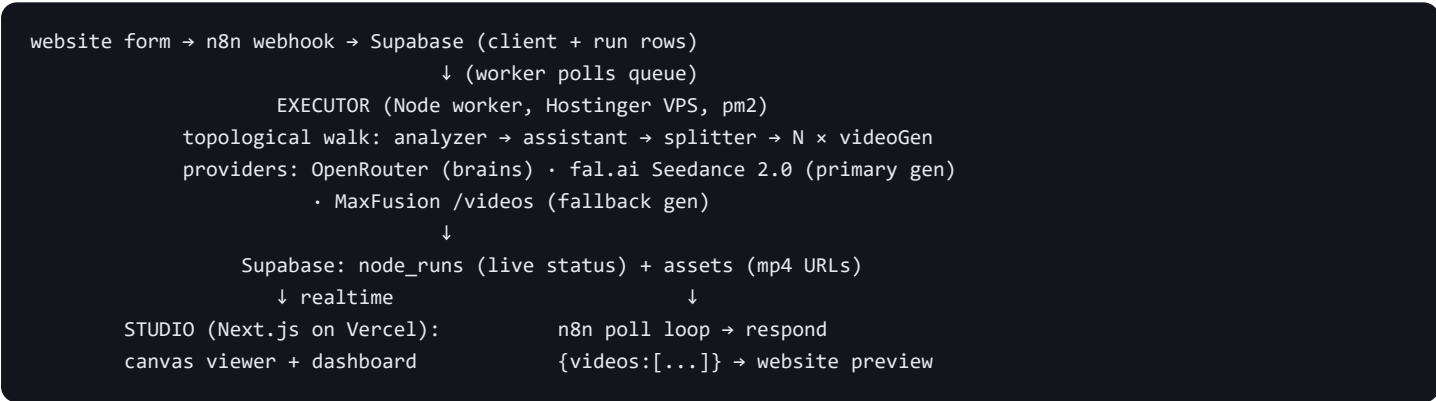
Pre-staged assets (reuse, do not redo)

- **Master prompts extracted clean via API** (not the mojibake .md copies): image analyzer (5,559 chars), video analyzer (8,645), ECOM UGC assistant (5,119) + the API-adaptation block (single-scene, embed product description, JSON output). Paste-ready in `Docs/Creative OS/n8n bodies/` .
- **MaxFusion MCP server** (`maxfusion-mcp/` , 10 tools) — registered in Claude Code; useful for ops/debug during the build.
- API knowledge base: `Docs/Creative OS/MaxFusion API - n8n Integration Guide.md` .
- 9 compressed sample ads in `Creative OS/videos/web/` for seeding the dashboard demo.

Landmines already hit (workaround stated — do not re-trip)

Landmine	Workaround
S3 upload sent base64 <code>text</code> as the file → analyzer "Every item failed to analyze"	Decode to bytes first; multipart field <code>file</code> must be binary and last
<code>POST /files</code> has no <code>.url</code>	Build public URL from <code>filename</code>
<code>run-bulk</code> looks like it works (returns <code>queued</code>)	It never executes headless — own executor only
<code>/videos</code> ignores unknown fields silently	Validate outbound body against a literal allowlist of the 5 fields
OpenRouter wraps JSON in fences	Strip ``json fences before parse
Spurious <code>content_policy_violation</code> on audio	Auto-retry the identical request once
Aspect ratios	Underscore style (<code>9_16</code>), never <code>9:16</code>
Vercel serverless timeout (10–60s) vs 3–5 min video jobs	Executor is a long-running worker on the Hostinger VPS , not a Vercel function

3. Architecture



Stack: Next.js 15 (App Router) + `@xyflow/react` (React Flow — native compatibility with MaxFusion `canvas_data` , so all 34 existing flows import unchanged) + Supabase (Postgres, storage, realtime, simple team auth) + Node executor worker on the VPS (pm2) + existing n8n as the public webhook edge.

DB schema (5 tables): `clients(id, name, website, created_at)` · `flows(id, client_id nullable, name, canvas jsonb, is_template bool, source_concept text)` · `runs(id, flow_id, client_id, status, requested_by, created_at, finished_at)` · `node_runs(run_id, node_id, status, output jsonb, error, cost, started_at, finished_at)` · `assets(id, run_id, node_id, type, url, thumb_url, duration_s)` .

Node types v1 (executor + canvas renderers): `imageInput` , `textPrompt` , `contentAnalyzer` (OpenRouter vision + image master prompt), `assistant` (OpenRouter + flow's systemPrompt), `splitter` (parse assistant JSON array → fan out), `videoGenerator` (provider-abstracted). `videoEditor` (ffmpeg concat on VPS) is v1.5.

Provider abstraction: `generateVideo(scene, cfg)` with `PROVIDER_ORDER=fal,maxfusion` env. fal.ai runs the same Seedance 2.0 cheaper (Eric's directive); MaxFusion path is already proven and stays as fallback. If the fal key isn't provisioned by build time, ship with `maxfusion` primary and the fal adapter behind the same interface with a mocked test.

4. Permissions / gotchas

- All secrets (**MaxFusion** `mfsk_` , **OpenRouter** `sk-or-` , **fal**, **Supabase service key**) live only in the worker's env and n8n credentials — never in the Next.js client bundle, never committed. Both the mfsk and OpenRouter keys were pasted in chats this week → **rotate both** when wiring the executor.

- Supabase Row Level Security: dashboard behind team login (magic link); `assets` bucket public-read (website must play videos without auth).
- CORS: studio API routes must allow the marketing site origin (`creative-os-blue.vercel.app`).
- Do not touch the live marketing site except the final webhook behavior (it already renders `{videos:[...]}` — keep that contract).
- VPS deploy needs only Node 20+ and pm2; document exact commands in README (no Docker assumption).

5. Tunable decisions (defaults chosen — all changeable in `studio.config.ts` / `env`)

Decision	Default
Generation provider order	<code>fal, maxfusion</code> (env <code>PROVIDER_ORDER</code>)
Video model / duration / ratio / resolution	<code>seedance-2.0</code> / 15s / <code>9_16</code> / test <code>480p</code> , deliver <code>1080p</code>
Variations per run (Ricardo pattern)	1 (website), 4 (studio "concept batch" button)
Job poll interval / run timeout	10s / 12 min
Content-policy auto-retry	1
Analyzer/assistant model	<code>anthropic/claude-sonnet-4.6</code> , max_tokens 8000
Seed templates	one per website Concept dropdown value, cloned from the imported MaxFusion flows
Studio branding	Poppins, purple #6B48FF, teal #00E5CC, navy #003F6A (match creativeadbundance.com)

6. Deliverables (file-by-file)

```
creative-os-studio/
├── PRD.md                               ← this document
├── supabase/schema.sql                  ← 5 tables + RLS + realtime publication
├── scripts/
│   ├── import-maxfusion.ts             ← pull all 34 flows + runtime outputs → flows/assets seed
│   └── seed-templates.ts               ← map Concept dropdown values → template flows
├── worker/                             ← THE MOAT (build first)
│   ├── executor.ts                    ← polls runs queue, topo-walks graph, writes node_runs live
│   ├── nodes/{analyzer,assistant,splitter,videoGen}.ts
│   ├── providers/{openrouter,maxfusion,fal}.ts
│   ├── lib/{mfApi,retry,fences}.ts     ← allowlisted /videos body, fence-strip, policy-retry
│   └── pm2.config.cjs · .env.example · README-deploy.md
├── app/ (Next.js studio, Vercel)
│   ├── (dashboard)/clients/page.tsx    ← clients → runs → video gallery (thumbs, download)
│   ├── runs/[id]/page.tsx              ← run detail + canvas live view
│   ├── canvas/[flowId]/page.tsx        ← React Flow viewer, MaxFusion JSON in/out,
│   │                                   node status colors via Supabase realtime,
│   │                                   editable input-node fields, Save-as-Template
│   ├── api/{runs,templates,clone}/route.ts
│   ├── components/nodes/*.tsx          ← 6 node renderers, MaxFusion-style dark cards
│   └── lib/{db,mf-format}.ts
└── n8n/creativeos-webhook.json         ← webhook → insert run → poll Supabase → respond {videos}
```

7. SUCCESS CRITERIA (numbered, testable — run each, don't vibe them)

1. `scripts/import-maxfusion.ts` imports ≥ 30 flows; **"ugc 15 seconds"** opens in the canvas with every node, edge, and master prompt intact (spot-check the 5,119-char assistant prompt).
 2. POST the real v3 payload (pillow test image) to the webhook: with **zero MaxFusion browser tabs open anywhere**, a ≥ 12 s mp4 exists in Supabase, appears in the dashboard, and returns to the website within 8 minutes.
 3. During (2), the run's canvas page shows nodes transitioning `queued → running → done` live (Supabase realtime), and survives a page refresh mid-run.
 4. Provider swap is env-only: the same run succeeds with `PROVIDER_ORDER=maxfusion`; the fal adapter passes its mocked contract test (live fal run too if the key exists by then).
 5. **Template loop**: save the completed client flow as a template → clone it for a second client with a different image + website → second run succeeds with no prompt edits.
 6. **Ricardo batch**: one studio-triggered run with `variations=4` produces 4 distinct-concept mp4s (480p allowed) under one run id.
 7. Forced failure (bad model id) → node marked `error` with the API's message, run `failed`, website receives a friendly JSON error — no infinite loop, no hang.
 8. A teammate follows README from zero → studio running locally + worker deployed to the VPS in under 30 minutes; `git grep` finds no secret in the repo.
 9. The executor codebase contains **zero calls to** `run-bulk`.
 10. Dashboard is usable at 375px width (Eric reviews from his phone).
-

8. Guardrails

- Build the **worker first**, canvas second, polish last. The website must work through the executor before any canvas pixel is drawn.
 - Reuse the staged master prompts verbatim from `Docs/Creative 05/n8n bodies/` — do not rewrite the designer's prompts.
 - Do not re-probe the MaxFusion API — every fact needed is in §2. Especially: never rely on `run-bulk`, never send `/videos` fields outside the 5-field allowlist.
 - Test generations at **480p, cap 10 videos** total during the build; 1080p only for the final acceptance run.
 - No new paid services beyond Supabase free tier and the existing keys (fal key pending — human task).
 - Don't modify the live marketing site except to keep its existing response contract working.
 - v1 canvas editing = input-node fields + save-as-template only. No drag-drop node creation yet.
-

9. Fable 5 kickoff prompt (paste verbatim after switching models)

```
/goal Build Creative OS Studio per the PRD.
```

Read creative-os-studio/PRD.md in full – it is the complete spec with live-verified API facts, staged assets, landmines, and Success Criteria. Build until ALL 10 success criteria pass. Run each one – do not stop at "should work."

Restated so you don't miss them:

- The executor is a long-running Node worker on the Hostinger VPS – NEVER a Vercel serverless function (video jobs outlive their timeout).
- NEVER use MaxFusion /flows/{id}/run-bulk – it cannot execute headlessly. Generation goes through POST /videos (allowlist exactly: video_model, prompt, duration, aspect_ratio, resolution) or the fal adapter.
- Reuse the staged master prompts in "Docs/Creative OS/n8n bodies/" and the import script's flows – do not rewrite prompts, do not re-probe the API.
- Website contract is frozen: flat v3 payload in, {videos:[urls]} out.
- Test at 480p, max 10 generations; strip ```json fences from every OpenRouter response; retry content_policy_violation exactly once.
- When done: run the full acceptance flow (criterion 2) live with me and walk me through the VPS deploy + key rotation steps.

Prepared by Claude (Fable 5) with Carl Sajol — Creative Abundance, July 4, 2026. Facts verified against live systems on Jul 3–4; see [Docs/Creative OS/MaxFusion API - n8n Integration Guide.md](#) for the full API evidence trail.